

JAVA INTERVIEW PREPARATION

CHAPTER 47

DATABASE TRANSACTIONS, ISOLATION, AND LOCKING

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Database Transactions, Isolation, and Locking: Preserving Invariants Under Concurrency

Senior / Lead Java Developer Reading Chapter

A transaction is not merely a Spring annotation. It is a database protocol that groups reads and writes into one outcome while concurrent transactions operate on the same state. Senior reasoning begins with the business invariant and a concrete concurrent schedule, then selects constraints, isolation, optimistic versions, or locks that make every allowed execution safe.

ACID as Operational Guarantees

Atomicity means the transaction's database effects commit together or roll back. Consistency means committed state satisfies the invariants actually enforced by constraints and correct application logic. Isolation limits what concurrent transactions can observe. Durability means a successful commit survives according to the database's persistence and replication guarantees.

```
begin
  debit account A
  credit account B
commit -> both database writes become one committed outcome
```

ACID does not automatically include a remote payment call or broker publication. It also does not prove a missing business constraint. The database preserves rules it has been told to enforce.

Autocommit and Transaction Scope

In autocommit mode, each statement normally becomes its own transaction. A read followed by a write is therefore not atomic merely because the Java method executed sequentially.

```
SELECT available FROM inventory WHERE sku = 'X'; -- transaction 1
UPDATE inventory SET available = available - 1; -- transaction 2
```

Define the transaction around one business decision. Keep it short: do not wait for user input, HTTP calls, message replies, or long computation while retaining connections, snapshots, or locks. Transaction scope determines contention and failure recovery as much as isolation level does.

Concurrency Phenomena

A dirty read observes uncommitted data. A non-repeatable read sees one row change between reads. A phantom changes the set matching a predicate. Lost update overwrites another committed change. Write skew lets transactions update different rows after reading a shared condition, violating an aggregate rule.

```
Invariant: at least one doctor remains on call

T1 reads Alice=on, Bob=on
T2 reads Alice=on, Bob=on
T1 sets Alice=off
T2 sets Bob=off
both commit -> invariant broken
```

The classic ANSI phenomena do not describe every anomaly. Reason from schedules and the actual database's MVCC or locking implementation rather than relying only on an isolation-level name.

Read Committed

Read Committed commonly prevents dirty reads but allows each statement to see a newer committed snapshot. Two SELECTs in one transaction may return different values. It offers good concurrency and is a common default.

For simple atomic changes, express the invariant in one statement:

```
update inventory
set available = available - :quantity
where sku = :sku and available >= :quantity;
```

One affected row means success; zero means insufficient stock or absence. This is safer than read-check-write and often needs no stronger isolation.

Repeatable Read and Snapshot Isolation

Repeatable Read guarantees vary by database. In PostgreSQL, a transaction sees a stable snapshot and must be prepared for update conflicts, while serialization anomalies can still occur. Other engines use different combinations of locks and snapshots.

A stable snapshot prevents many anomalies but does not automatically protect a rule spanning multiple rows. Write skew can remain when two transactions update different rows. Use Serializable, explicit locking of a shared guard row, or redesign the invariant into a constraint or atomic statement.

Serializable

Serializable means committed concurrent transactions have an outcome equivalent to some serial ordering. Databases may implement this through strict locking or by detecting dangerous dependency structures and aborting a transaction.

Serialization failure is an expected concurrency outcome. Retry the entire transaction from the beginning in a fresh transaction, using newly read state. Bound attempts and apply jitter when contention is high.

```
attempt begins -> read current state -> decide -> write -> commit
serialization failure
new attempt    -> reread state      -> decide again
```

Do not retry only the final statement or reuse managed entities from the aborted transaction. Remote side effects inside the attempt must be idempotent or moved outside through durable intent.

Optimistic Locking

Optimistic locking assumes conflicts are uncommon. A version column makes the update conditional on the version originally read.

```
update orders
set status = :status, version = version + 1
where id = :id and version = :expected;
```

Zero updated rows signals a conflict. JPA's `@Version` supports this pattern. The application can reject with a conflict, reread and merge, or retry a safe command. Blind retry is wrong when the user must reconsider another person's change.

Optimistic locking detects competing updates to the same versioned entity. It does not automatically protect a cross-row invariant or external side effect.

Pessimistic Row Locks

`SELECT ... FOR UPDATE` acquires row-level locks so competing writers wait or fail according to timeout policy.

```
select * from accounts where id in (:from, :to)
for update;
```

Pessimistic locking is useful for short, highly contended critical sections where waiting is preferable to repeated optimistic failure. It holds database resources and can create deadlocks. Lock all required rows in a stable order and keep the transaction bounded.

Locking existing rows does not always protect the absence of a row or a predicate. Unique constraints, exclusion constraints, range locks, guard rows, or Serializable isolation may be needed.

Constraints Are Concurrency Controls

Database constraints close races no application pre-check can close.

```
create unique index ux_active_booking
on booking(resource_id, slot_start)
where status = 'ACTIVE';
```

Two transactions can both see no booking; only one can commit the unique key. Translate the constraint violation into a stable domain conflict. Keep the pre-check for friendly feedback, but treat the constraint as final authority.

Foreign keys, NOT NULL, CHECK, uniqueness, and exclusion constraints encode durable invariants across all writers, including scripts and other services with access.

Deadlocks and Lock Timeouts

A deadlock forms when transactions wait in a cycle.

```
T1 holds account A, waits for B
T2 holds account B, waits for A
```

Databases detect a deadlock and abort a participant. Prevention includes consistent lock order, short transactions, correct indexes, and avoiding user or network waits. Lock timeout bounds waiting but does not diagnose the cycle.

Capture the deadlock graph, SQL, transaction age, and access order. Retry only the entire idempotent transaction and alert on sustained rates, which often indicate a design or traffic change.

MVCC, Long Transactions, and Vacuum Pressure

MVCC keeps row versions so readers and writers can proceed concurrently. Old versions remain needed while transactions can still see them. A long-lived transaction can therefore prevent cleanup, increase table and index bloat, retain undo or version storage, and amplify replication lag.

Monitor transactions that are idle while open, oldest snapshot age, lock waits, dead tuples or undo growth, and connection usage. A read-only transaction can still be operationally expensive if it holds an old snapshot for hours.

Spring and JPA Boundaries

Spring's `@Transactional` commonly creates a proxy-managed logical scope. The database connection and physical transaction are bound when the proxied call executes. Self-invocation can bypass the proxy, and new threads do not inherit an imperative transaction.

JPA flush sends pending SQL but does not commit. Constraint or optimistic-lock failures may appear at flush before method return. A transaction can also be marked rollback-only and fail at outer commit.

Do not catch a database exception inside the transaction and continue using the same persistence context. After many SQL failures the transaction is unusable; let it roll back, translate at the boundary, and start a fresh attempt when policy permits.

Read-Only and Replica Transactions

Read-only is often an optimization hint, not an authorization boundary. It does not universally guarantee that no write reaches every database or side-effecting function.

Replica reads introduce lag. A client can write to the primary and immediately miss the row on a replica. Define read-your-write routing, version tokens, bounded staleness, or user-visible processing state. Do not call eventual replication a transaction-isolation issue; it is a separate consistency boundary.

Choosing Concurrency Control from the Invariant

Use a decision process rather than selecting one isolation level for the whole application. First ask whether a declarative constraint can express the rule. A unique or exclusion constraint is both precise and independent of application access path. Next ask whether one conditional DML statement can combine validation and mutation atomically. Only then introduce explicit read-modify-write coordination.

For a same-row edit with rare contention, an optimistic version provides high concurrency and visible conflict. For a short reservation on a known row with frequent contention, pessimistic locking can reduce wasted retries. For a predicate or cross-row invariant, consider Serializable isolation, an explicitly locked guard row representing the invariant, or a schema that turns the rule into a unique key.

Document the conflict response. A machine workflow may retry after rereading. A human editing stale customer data should usually see a conflict rather than silently overwrite. A financial operation may move to reconciliation when remote outcome is uncertain. Concurrency policy includes user semantics, not only database syntax.

Advisory and Application Locks

Some databases provide advisory locks identified by an application-defined key. They can coordinate work that has no convenient row, such as one scheduler task per tenant. Because the database does not infer the protected data, every participant must use the same key and protocol.

```
hash("invoice-run", tenant-17) -> advisory lock key
worker A acquires -> runs bounded job
worker B cannot acquire -> skips or waits by policy
```

Prefer transaction-scoped advisory locks when available so rollback or connection cleanup releases them. Session-scoped locks can survive longer than intended in pooled connections. Hash collisions, inconsistent key construction, and code paths that forget the lock can violate assumptions.

Distributed lock services have the same fundamental requirement: a lease expiring does not stop the old owner. For external side effects, use fencing tokens or conditional versions so a stale owner cannot commit after a new owner takes over. A lock alone is not proof that work is still exclusively authorized.

Transaction Retry Design

Place retry outside the transaction interceptor so each attempt gets a new physical transaction and persistence context. Classify only known transient SQL states such as serialization failure or deadlock victim; constraint violations and invalid data are normally deterministic.

The retrying operation must be safe as a whole. Database changes roll back, but an HTTP call, email, or message sent before the abort may already be accepted. Record durable intent in the local transaction and perform external publication afterward, or reuse a stable idempotency key on every attempt.

Track attempt count and final outcome separately. A successful request after three serialization failures is a success for the caller but evidence of contention for capacity and schema design.

Testing Concurrent Correctness

Use the production database engine and separate connections. Coordinate two transactions with latches so the intended schedule is created deliberately. Assert the invariant and expected conflict, not merely final happy-path values.

Test lost update, write skew, uniqueness races, lock timeout, deadlock victim handling, serialization retry, and optimistic conflict. A test method wrapped in one transaction cannot model two actors. Flush and clear JPA state when database behavior must be observed.

Production Diagnosis

Measure connection acquisition, transaction duration, idle-in-transaction time, commit and rollback outcome, deadlocks, serialization failures, lock waits, and rows touched. Correlate request traces with database sessions and query fingerprints.

During an incident, distinguish CPU-heavy SQL from lock waiting and connection starvation. Killing a blocker may restore service but destroys evidence and can roll back large work. Capture safe lock and session state first, then mitigate.

Common Senior-Level Traps

Common traps include assuming Repeatable Read means the same thing on every database, using read-check-write without an atomic predicate or constraint, holding locks across HTTP calls, retrying inside an already failed transaction, and treating optimistic locking as protection for every invariant.

Another trap is increasing isolation globally without measuring aborts, blocking, and throughput. Select concurrency control per invariant and query pattern.

A Strong Interview Answer

I start from the invariant and write the concurrent schedule that could violate it. I prefer database constraints and single conditional statements when they express the rule. For same-row conflicts I use an optimistic version when contention is low, and short ordered pessimistic locks when contention is high. Cross-row rules may need Serializable isolation, a guard row, or a schema redesign. Isolation names are database-specific, so I verify the target engine. Serialization and deadlock failures retry the entire idempotent transaction in a fresh context. I keep transactions short, avoid remote work inside them, test with separate real connections, and observe lock wait, snapshot age, connection acquisition, and rollback cause.

Interview-Ready Summary

Transactions preserve local database invariants only when scope, SQL, constraints, and concurrency control are correct. MVCC improves concurrency but creates snapshots and version-retention costs. Isolation must be understood through actual engine behavior.

Conditional writes and constraints are often stronger than application pre-checks. Optimistic versions detect conflicts; pessimistic locks serialize selected work; Serializable permits safe outcomes or aborts. Every expected abort needs bounded recovery and production evidence.

Revision Notes

- Define the business invariant before choosing isolation.
- ACID applies to the participating transactional database resource.
- Autocommit makes separate statements separate transactions.
- Dirty read, non-repeatable read, phantom, lost update, and write skew differ.
- Isolation-level behavior varies by database implementation.
- Prefer one conditional update over read-check-write where possible.
- PostgreSQL Read Committed uses a statement-level committed snapshot.
- Snapshot stability does not eliminate every serialization anomaly.
- Serializable may abort transactions that must be fully retried.
- Retry in a fresh transaction using newly read state.
- JPA `@Version` implements optimistic conflict detection.
- Optimistic locking does not protect every cross-row invariant.
- Pessimistic locks suit short, highly contended critical sections.
- Acquire multiple locks in a consistent order.
- Constraints are durable concurrency controls.
- Locking rows may not protect predicates or missing rows.
- Deadlock victims and lock timeouts are different outcomes.
- Long transactions retain versions and delay cleanup.
- Flush sends SQL; commit makes the transaction durable.
- Test concurrency with separate real database connections.
- Observe transaction duration, lock wait, aborts, and snapshot age.